

Edit Listing Image Guide



You said:

ChatGPT said:

To edit a listing image in your WanderLust project, you need to:

1. Upload a new image in the Edit form
2. Save the new image URL + filename in MongoDB
3. Replace the old image when updating the listing

Here's the complete flow.

1. Edit Form (**edit.ejs**)

Add file input inside your edit form.

```
HTML<form method="POST" action="/listings/<%= listing._id %>?_method=PUT"
enctype="multipart/form-data">

  <input
    type="text"
    name="listing[title]"
    value="<%= listing.title %>"
  />

  <!-- Old Image -->
  

  <!-- New Image Upload -->
  <input
    type="file"
    name="listing[image]"
  />

  <button>Edit</button>
</form>
```

2. Route (**routes/listing.js**)

Use multer middleware in update route.

```
JavaScriptrouter.put(
  "/:id",
  isLoggedIn,
  isOwner,
  upload.single("listing[image]"),
  wrapAsync(listingController.updateListing)
);
```

3. Controller (**controllers/listings.js**)

Update image if user uploads a new one.

```
JavaScriptmodule.exports.updateListing = async (req, res) => {  
  
  let { id } = req.params;  
  
  let listing = await Listing.findByIdAndUpdate(  
    id,  
    { ...req.body.listing }  
  );  
  
  // If new image uploaded  
  if (typeof req.file !== "undefined") {  
  
    let url = req.file.path;  
    let filename = req.file.filename;  
  
    listing.image = { url, filename };  
  
    await listing.save();  
  }  
  
  req.flash("success", "Listing Updated!");  
  
  res.redirect(`/listings/${id}`);  
};
```

4. Multer + Cloudinary Setup

Your `cloudConfig.js`

```
JavaScriptconst cloudinary = require("cloudinary").v2;
const { CloudinaryStorage } = require("multer-storage-cloudinary");

cloudinary.config({
  cloud_name: process.env.CLOUD_NAME,
  api_key: process.env.CLOUD_API_KEY,
  api_secret: process.env.CLOUD_API_SECRET,
});

const storage = new CloudinaryStorage({
  cloudinary,
  params: {
    folder: "WanderLust_DEV",
    allowedFormats: ["png", "jpg", "jpeg"],
  },
});

module.exports = {
  cloudinary,
  storage,
};
```

5. Multer Setup

Inside route file:

```
JavaScriptconst multer = require("multer");
const { storage } = require("../cloudConfig.js");

const upload = multer({ storage });
```

6. Listing Schema

Your image object should look like this:

```
JavaScriptimage: {
  url: String,
  filename: String,
},
```

7. Important Form Attribute

Without this, image upload will NOT work:

```
HTMLenctype="multipart/form-data"
```

Final Working Flow

User edits listing → uploads new image → multer uploads to Cloudinary → new URL stored in MongoDB → updated image displayed.

If you want, I can also show:

- Delete old Cloudinary image automatically
- Preview image before upload
- Multiple image upload
- Drag & drop image upload
- Image validation (size/type)
- Full professional edit listing page styling

You said:

image preview for edit page

ChatGPT said:

Add live image preview using JavaScript.

Edit Page (**edit.ejs**)

```
HTML<form
  method="POST"
  action="/listings/<%= listing._id %>?_method=PUT"
  enctype="multipart/form-data"
>

<!-- Current Image -->
<div>
  <p>Current Image</p>

  
</div>

<br>

<!-- Upload New Image -->
<input
  type="file"
  name="listing[image]"
  id="imageUpload"
  accept="image/*"
/>

<br><br>

<button>Edit Listing</button>

</form>

<!-- JavaScript -->
<script>

const imageUpload = document.getElementById("imageUpload");
const previewImage = document.getElementById("previewImage");

imageUpload.addEventListener("change", function (event) {

  const file = event.target.files[0];

  if (file) {

    previewImage.src = URL.createObjectURL(file);
```

```
    }  
  });  
  
</script>
```

How It Works

- Old image shown initially
- User selects new image
- JavaScript instantly updates preview
- No page reload needed

Better Styling (Optional)

```
CSS#previewImage{  
  width:300px;  
  height:200px;  
  object-fit:cover;  
  border-radius:12px;  
  margin-bottom:10px;  
}
```

Result

✅ Existing image visible ✅ New image preview instantly ✅ Professional UX like modern apps

You can also add:

- Drag & drop upload
- Remove selected image button
- Multiple image previews
- Image size validation
- Bootstrap styled preview card
- Loading spinner during upload

You said:

getting started with maps

ChatGPT said:

To get started with maps in your WanderLust project, you are probably using Mapbox.

Here's the complete beginner-to-working setup.

1. Install Mapbox Packages

Run:

```
Bashnpm install @mapbox/mapbox-sdk  
npm install mapbox-gl
```

2. Get Mapbox Access Token

Go to:

[Mapbox Official Website](#)

Create account → Dashboard → Access Tokens → Copy Default Public Token.

3. Add Token in .env

```
envMAP_TOKEN=your_mapbox_token_here
```

Example:

```
envMAP_TOKEN=pk.eyJ1IjoiYWJjZCIsImEiOiJja3...
```


4. Configure Mapbox (**controllers/listings.js**)

```
JavaScriptconst mbxGeocoding = require("@mapbox/mapbox-sdk/services/geocoding");

const mapToken = process.env.MAP_TOKEN;

const geocodingClient = mbxGeocoding({
  accessToken: mapToken,
});
```

5. Get Coordinates from Location

Inside Create Listing Controller:

```
JavaScriptmodule.exports.createListing = async (req, res) => {

  let response = await geocodingClient
    .forwardGeocode({
      query: req.body.listing.location,
      limit: 1,
    })
    .send();

  let newListing = new Listing(req.body.listing);

  newListing.owner = req.user._id;

  // Save coordinates
  newListing.geometry = response.body.features[0].geometry;

  await newListing.save();

  req.flash("success", "New Listing Created!");

  res.redirect("/listings");
};
```

6. Add Geometry in Schema

models/listing.js

```
JavaScriptgeometry: {  
  type: {  
    type: String,  
    enum: ["Point"],  
    required: true,  
  },  
  coordinates: {  
    type: [Number],  
    required: true,  
  },  
},
```

7. Save Existing Listings with Coordinates

MongoDB stores:

```
JavaScriptgeometry: {  
  type: "Point",  
  coordinates: [longitude, latitude]  
}
```

Example:

```
JavaScriptcoordinates: [72.8777, 19.0760]
```

8. Add Map on Show Page

Inside `show.ejs`

```
HTML<link
href="https://api.mapbox.com/mapbox-gl-js/v3.1.2/mapbox-gl.css"
rel="stylesheet"
/>

<div id="map"></div>

<style>
#map{
  height:400px;
  width:100%;
  border-radius:15px;
}
</style>

<script src="https://api.mapbox.com/mapbox-gl-js/v3.1.2/mapbox-gl.js"></script>
```

9. Create Map JS File

public/js/map.js

```
JavaScriptmapboxgl.accessToken = mapToken;

const map = new mapboxgl.Map({
  container: 'map',

  style: 'mapbox://styles/mapbox/streets-v12',

  center: listing.geometry.coordinates,

  zoom: 9
});

const marker = new mapboxgl.Marker()
  .setLngLat(listing.geometry.coordinates)
  .addTo(map);
```

10. Pass Data from EJS

Inside `show.ejs`

```
HTML<script>

  const listing = <%- JSON.stringify(listing) %>;

  const mapToken = "<%= process.env.MAP_TOKEN %>";

</script>

<script src="/js/map.js"></script>
```

Final Result

✅ User enters location ✅ Mapbox converts location → coordinates ✅
Coordinates stored in MongoDB ✅ Map displayed on listing page ✅ Marker
shown automatically

Important Packages

Package	Use
@mapbox/mapbox-sdk	Geocoding
mapbox-gl	Frontend Map
dotenv	Environment Variables

Common Errors

1. Cannot create client without access token

Fix:

```
envMAP_TOKEN=your_token
```

AND:

```
JavaScriptrequire("dotenv").config();
```

2. Map Not Visible

Fix:

```
CSS#map{  
  height:400px;  
}
```

3. Coordinates Undefined

Check:

```
JavaScriptconsole.log(response.body.features);
```

Architecture Flow

```
graph TD; A[User Location] --> B[Mapbox Geocoding API]; B --> C[Coordinates Generated]; C --> D[MongoDB Stores Geometry]; D --> E[Mapbox GL Displays Map];
```

User Location
↓
Mapbox Geocoding API
↓
Coordinates Generated
↓
MongoDB Stores Geometry
↓
Mapbox GL Displays Map

You can next learn:

- Cluster maps
- Search nearby listings
- Interactive popups
- Multiple markers
- Map dark mode
- Live location
- Nearby restaurants/hotels
- Full Airbnb-style maps
- Geospatial queries in MongoDB

You said:

Show more

ChatGPT said:

Your `show.ejs` looks mostly correct. You only need a few things to make the map work properly.

1. Add Mapbox CSS

Add this near the top of file:

```
HTML<link
href="https://api.mapbox.com/mapbox-gl-js/v3.1.2/mapbox-gl.css"
rel="stylesheet"
/>
```

Place it below:

```
ejs<% layout("/layouts/boilerplate") %>
```

2. Add Mapbox JS

Add BEFORE your `map.js` file:

```
HTML<script src="https://api.mapbox.com/mapbox-gl-js/v3.1.2/mapbox-gl.js">
</script>
```

3. Final Bottom Section

Replace your bottom section with this:

```
HTML<div class="col-6 offset-3 mb-3">
  <h3>Where you'll be</h3>
  <div id="map"></div>
</div>

<script src="https://api.mapbox.com/mapbox-gl-js/v3.1.2/mapbox-gl.js"></script>

<script src="/js/map.js"></script>
```

4. Add Map CSS

Inside your `style.css`

```
CSS#map {  
  height: 400px;  
  width: 100%;  
  border-radius: 15px;  
}
```

Without height → map will NOT appear.

5. Create `public/js/map.js`

```
JavaScriptmapboxgl.accessToken = mapToken;  
  
const map = new mapboxgl.Map({  
  container: "map",  
  
  style: "mapbox://styles/mapbox/streets-v12",  
  
  center: listing.geometry.coordinates,  
  
  zoom: 9,  
});  
  
const marker = new mapboxgl.Marker({ color: "red" })  
  .setLngLat(listing.geometry.coordinates)  
  
  .setPopup(  
    new mapboxgl.Popup({ offset: 25 })  
    .setHTML(  
      `

#### ${listing.title}</h4> <p>Exact location provided after booking</p>` ) ) .addTo(map);


```

6. Make Sure Geometry Exists

Your listing document must contain:

```
JavaScriptgeometry: {  
  type: "Point",  
  coordinates: [longitude, latitude]  
}
```

Example:

```
JavaScriptcoordinates: [72.8777, 19.0760]
```

7. Most Common Error

Error:

```
BashCannot read properties of undefined (reading 'coordinates')
```

Fix:

Your listing does NOT contain geometry.

Add this in create controller:

```
JavaScriptnewListing.geometry = response.body.features[0].geometry;
```

8. Full Working Flow

```
graph TD  
  A[Location Input] --> B[Mapbox Geocoding]  
  B --> C[Coordinates Generated]  
  C --> D[MongoDB Stores Geometry]  
  D --> E[Mapbox Displays Map]
```


Your File Structure

```
public
├── js
│   └── map.js
views
├── listings
│   └── show.ejs
```

Final Result

✅ Interactive Map ✅ Red Marker ✅ Popup Message ✅ Dynamic Coordinates ✅ Airbnb-style Listing Page

You said:

Show more

ChatGPT said:

Your `show.ejs` is missing only ONE important thing:

You did NOT include the Mapbox CDN files.

Add these two lines.

1. Add Mapbox CSS

Place BELOW:

```
ejs<% layout("/layouts/boilerplate") %>
```

Add:

```
HTML<link
  href="https://api.mapbox.com/mapbox-gl-js/v3.1.2/mapbox-gl.css"
  rel="stylesheet"
/>
```

2. Add Mapbox JS

Place ABOVE:

```
HTML<script src="/js/map.js"></script>
```

Add:

```
HTML<script src="https://api.mapbox.com/mapbox-gl-js/v3.1.2/mapbox-gl.js">
</script>
```

3. Final Bottom Section Should Be

```
HTML<div class="col-6 offset-3 mb-3">
  <h3>Where you'll be</h3>
  <div id="map"></div>
</div>

<script src="https://api.mapbox.com/mapbox-gl-js/v3.1.2/mapbox-gl.js"></script>

<script src="/js/map.js"></script>
```

4. Add CSS

In `public/css/style.css`

```
CSS#map{
  height:400px;
  width:100%;
  border-radius:20px;
}
```

5. Your `map.js`

Create:

`public/js/map.js`

```
JavaScriptmapboxgl.accessToken = mapToken;

const map = new mapboxgl.Map({
  container: "map",

  style: "mapbox://styles/mapbox/streets-v12",

  center: listing.geometry.coordinates,

  zoom: 9,
});

const marker = new mapboxgl.Marker({ color: "red" })

  .setLngLat(listing.geometry.coordinates)

  .setPopup(
    new mapboxgl.Popup({ offset: 25 })
      .setHTML(
        `<h4>${listing.title}</h4>
        <p>Exact location provided after booking</p>`
      )
  )

  .addTo(map);
```

6. MOST IMPORTANT

Your listing must contain:

```
JavaScriptgeometry: {
  type: "Point",
  coordinates: [longitude, latitude]
}
```

Otherwise this error comes:

```
BashCannot read properties of undefined (reading 'coordinates')
```

7. Fix for Old Listings

Old listings usually don't have geometry.

You must:

- Edit listing again OR
- Recreate listing OR
- Add geometry manually in MongoDB

Final Result

✅ Interactive Map ✅ Popup ✅ Marker ✅ Dynamic Coordinates ✅ Airbnb-style location section

You said:

ChatGPT said:

To create your **first map** in the WanderLust project using Mapbox, follow these simple steps.

1. Add Map Container

Inside `show.ejs`

```
HTML<div id="map"></div>
```

2. Add Mapbox CSS

At top of `show.ejs`

```
HTML<link
href="https://api.mapbox.com/mapbox-gl-js/v3.1.2/mapbox-gl.css"
rel="stylesheet"
/>
```

3. Add Mapbox JS

Before `map.js`

```
HTML<script src="https://api.mapbox.com/mapbox-gl-js/v3.1.2/mapbox-gl.js">
</script>
```

4. Pass Token from EJS

```
HTML<script>
  const mapToken = "<%= process.env.MAP_TOKEN %>";
</script>
```

5. Create **map.js**

Path:

```
public/js/map.js
```

Code:

```
JavaScriptmapboxgl.accessToken = mapToken;

const map = new mapboxgl.Map({

  container: "map",

  style: "mapbox://styles/mapbox/streets-v12",

  center: [72.8777, 19.0760],

  zoom: 9,

});
```

6. Connect **map.js**

Inside **show.ejs**

```
HTML<script src="/js/map.js"></script>
```

7. Add Map Height

Inside **style.css**

```
CSS#map{
  height:400px;
  width:100%;
}
```

Without height → map invisible.

8. Final Working Structure

```
HTML<link
href="https://api.mapbox.com/mapbox-gl-js/v3.1.2/mapbox-gl.css"
rel="stylesheet"
/>

<div id="map"></div>

<script>
  const mapToken = "<%= process.env.MAP_TOKEN %>";
</script>

<script src="https://api.mapbox.com/mapbox-gl-js/v3.1.2/mapbox-gl.js"></script>

<script src="/js/map.js"></script>
```

9. Output

You will see:

✅ Interactive Map ✅ Zoom Controls ✅ Dragging ✅ Real Streets Map

10. Add Marker (Optional)

Inside `map.js`

```
JavaScriptnew mapboxgl.Marker()

  .setLngLat([72.8777, 19.0760])

  .addTo(map);
```

Result

Now your first map is complete 🎉

You said:

ChatGPT said:

Geocoding means:

```
Location Name → Coordinates
```

Example:

```
"Mumbai" → [72.8777, 19.0760]
```

In WanderLust, Mapbox Geocoding converts user location input into latitude & longitude so the map can display that place.

1. Install Mapbox SDK

Run:

```
Bashnpm install @mapbox/mapbox-sdk
```

2. Add MAP Token

`.env`

```
envMAP_TOKEN=your_mapbox_token
```

3. Configure Geocoding

Inside controller file:

```
JavaScriptconst mbxGeocoding = require("@mapbox/mapbox-sdk/services/geocoding");

const mapToken = process.env.MAP_TOKEN;

const geocodingClient = mbxGeocoding({
  accessToken: mapToken,
});
```

4. Use Forward Geocoding

Inside Create Listing Controller:

```
JavaScriptmodule.exports.createListing = async (req, res) => {  
  
  let response = await geocodingClient  
    .forwardGeocode({  
      query: req.body.listing.location,  
      limit: 1,  
    })  
    .send();  
  
  console.log(response.body.features);  
  
};
```

5. What Happens

If user enters:

Delhi

Mapbox returns:

```
JavaScript[  
  {  
    geometry: {  
      coordinates: [77.1025, 28.7041],  
      type: "Point"  
    }  
  }  
]
```

6. Save Coordinates in MongoDB

```
JavaScriptnewListing.geometry = response.body.features[0].geometry;
```

Complete Example:


```
JavaScriptmodule.exports.createListing = async (req, res) => {

  let response = await geocodingClient
    .forwardGeocode({
      query: req.body.listing.location,
      limit: 1,
    })
    .send();

  const newListing = new Listing(req.body.listing);

  newListing.owner = req.user._id;

  // Save coordinates
  newListing.geometry = response.body.features[0].geometry;

  await newListing.save();

  req.flash("success", "New Listing Created!");

  res.redirect("/listings");
};
```

7. Add Geometry Schema

Inside `models/listing.js`

```
JavaScriptgeometry: {
  type: {
    type: String,
    enum: ["Point"],
  },

  coordinates: {
    type: [Number],
  },
},
```

8. MongoDB Stored Data

```
JavaScriptgeometry: {
  type: "Point",
  coordinates: [72.8777, 19.0760]
}
```

9. Use Coordinates in Map

Inside `map.js`

```
JavaScriptcenter: listing.geometry.coordinates
```

Marker:

```
JavaScript.setLngLat(listing.geometry.coordinates)
```

10. Flow Diagram

```
graph TD; A[User Enters Location] --> B[Mapbox Geocoding API]; B --> C[Coordinates Generated]; C --> D[Save in MongoDB]; D --> E[Display on Map];
```

11. Important Errors

Error 1

```
BashCannot create client without access token
```

Fix:

```
JavaScriptrequire("dotenv").config();
```

AND:

```
envMAP_TOKEN=your_token
```

Error 2

```
BashCannot read properties of undefined (reading 'geometry')
```

Fix:

Location not found.

Use:

```
JavaScriptconsole.log(response.body.features);
```

12. Geocoding Types

Forward Geocoding

```
Location → Coordinates
```

Example:

```
Mumbai → [72.8777,19.0760]
```

Reverse Geocoding

```
Coordinates → Location
```

Example:

```
[72.8777,19.0760] → Mumbai
```

Final Result

✓ User types location ✓ Coordinates generated automatically ✓ MongoDB stores geometry ✓ Map displays correct location ✓ Airbnb-style maps work properly

You said:

ChatGPT said:

To store coordinates in your WanderLust project using Mapbox geocoding, follow this complete flow.

1. Add Geometry Field in Schema

Inside `models/listing.js`

```
JavaScriptconst listingSchema = new Schema({  
  
  title: String,  
  
  location: String,  
  
  country: String,  
  
  geometry: {  
    type: {  
      type: String,  
      enum: ["Point"],  
      required: true,  
    },  
  
    coordinates: {  
      type: [Number],  
      required: true,  
    },  
  },  
  
});
```

2. Install Mapbox SDK

```
Bashnpm install @mapbox/mapbox-sdk
```

3. Configure Geocoding

Inside controller file:

```
JavaScriptconst mbxGeocoding = require("@mapbox/mapbox-sdk/services/geocoding");

const mapToken = process.env.MAP_TOKEN;

const geocodingClient = mbxGeocoding({
  accessToken: mapToken,
});
```

4. Store Coordinates While Creating Listing

Inside `controllers/listings.js`

```
JavaScriptmodule.exports.createListing = async (req, res) => {

  // Convert location → coordinates
  let response = await geocodingClient
    .forwardGeocode({
      query: req.body.listing.location,
      limit: 1,
    })
    .send();

  // Create listing
  let newListing = new Listing(req.body.listing);

  // Store geometry
  newListing.geometry = response.body.features[0].geometry;

  // Store owner
  newListing.owner = req.user._id;

  await newListing.save();

  req.flash("success", "New Listing Created!");

  res.redirect("/listings");
};
```

5. What Gets Stored

If user enters:

Mumbai

MongoDB stores:

```
JavaScriptgeometry: {  
  type: "Point",  
  
  coordinates: [72.8777, 19.0760]  
}
```

6. Understand Coordinates

```
[longitude, latitude]
```

NOT:

```
[latitude, longitude]
```

Correct:

```
JavaScript[72.8777, 19.0760]
```

7. Use Coordinates in Map

Inside `map.js`

```
JavaScriptcenter: listing.geometry.coordinates
```

Marker:

```
JavaScript.setLngLat(listing.geometry.coordinates)
```

8. Check Stored Data

Inside MongoDB Compass:

```
JavaScript{
  title: "Beach House",

  geometry: {
    type: "Point",

    coordinates: [
      72.8777,
      19.0760
    ]
  }
}
```

9. Important Console Check

Before saving:

```
JavaScriptconsole.log(response.body.features[0].geometry);
```

Output:

```
JavaScript{
  coordinates: [72.8777, 19.0760],
  type: "Point"
}
```

10. Common Error

Error

```
BashCannot read properties of undefined (reading 'geometry')
```

Fix:

Mapbox could not find location.

Check:

```
JavaScriptconsole.log(response.body.features);
```

11. Full Flow

```
graph TD; A[User enters location] --> B[Mapbox Geocoding]; B --> C[Coordinates generated]; C --> D[Store in MongoDB]; D --> E[Use in map.js]; E --> F[Map displayed];
```

User enters location
↓
Mapbox Geocoding
↓
Coordinates generated
↓
Store in MongoDB
↓
Use in map.js
↓
Map displayed

Final Result

✅ Location converted to coordinates ✅ Coordinates stored in MongoDB ✅
Dynamic maps work ✅ Marker appears automatically ✅ Airbnb-style map
integration complete

You said:

ChatGPT said:

You probably mean **Map Marker** in Mapbox.

A marker shows the exact location on the map.

Basic Marker

Inside `public/js/map.js`


```
JavaScriptmapboxgl.accessToken = mapToken;

const map = new mapboxgl.Map({

  container: "map",

  style: "mapbox://styles/mapbox/streets-v12",

  center: listing.geometry.coordinates,

  zoom: 9,
});

new mapboxgl.Marker()

  .setLngLat(listing.geometry.coordinates)

  .addTo(map);
```

Add Red Marker

```
JavaScriptnew mapboxgl.Marker({ color: "red" })

  .setLngLat(listing.geometry.coordinates)

  .addTo(map);
```

Add Popup

```
JavaScriptnew mapboxgl.Marker({ color: "red" })

  .setLngLat(listing.geometry.coordinates)

  .setPopup(
    new mapboxgl.Popup({ offset: 25 })
      .setHTML(
        `

#### ${listing.title}</h4> <p>Exact location provided after booking</p>` ) ) .addTo(map);


```

Complete **map.js**

```
JavaScriptmapboxgl.accessToken = mapToken;

const map = new mapboxgl.Map({

  container: "map",

  style: "mapbox://styles/mapbox/streets-v12",

  center: listing.geometry.coordinates,

  zoom: 10,
});

const marker = new mapboxgl.Marker({ color: "red" })

  .setLngLat(listing.geometry.coordinates)

  .setPopup(
    new mapboxgl.Popup({ offset: 25 })
      .setHTML(
        `<h4>${listing.title}</h4>
        <p>Welcome to WanderLust!</p>`
      )
  )

  .addTo(map);
```

Output

✅ Interactive Marker ✅ Popup Message ✅ Dynamic Location ✅

Professional Airbnb-style Map

Important

Your listing must contain:

```
JavaScriptgeometry: {
  type: "Point",
  coordinates: [72.8777, 19.0760]
}
```

Map Marker Flow

Coordinates
↓
Marker Created
↓
Placed on Map
↓
Popup Attached

Styling Popup (Optional)

```
CSS.mapboxgl-popup-content{  
  border-radius:10px;  
  padding:10px;  
}
```

Final Result

You now have:

✅ Map ✅ Marker ✅ Popup ✅ Dynamic Coordinates ✅ Modern UI Map
Experience

You said:

ChatGPT said:

A popup in Mapbox appears when the user clicks the marker.

It can show:

- Listing title
- Price
- Description
- Message

- HTML content

Basic Popup

Inside `public/js/map.js`

```
JavaScriptconst popup = new mapboxgl.Popup({  
  offset: 25,  
})  
  
.setText("Welcome to WanderLust!");
```

Attach Popup to Marker

```
JavaScriptnew mapboxgl.Marker()  
  
.setLngLat(listing.geometry.coordinates)  
  
.setPopup(popup)  
  
.addTo(map);
```

Complete Example

```
JavaScriptmapboxgl.accessToken = mapToken;

const map = new mapboxgl.Map({

  container: "map",

  style: "mapbox://styles/mapbox/streets-v12",

  center: listing.geometry.coordinates,

  zoom: 10,
});

const popup = new mapboxgl.Popup({
  offset: 25,
})

.setHTML(
  `<h4>${listing.title}</h4>
  <p>Exact location provided after booking</p>`
);

new mapboxgl.Marker({ color: "red" })

  .setLngLat(listing.geometry.coordinates)

  .setPopup(popup)

  .addTo(map);
```

Better Popup Design

```
JavaScript.setHTML(
  `
  <div>
    <h4>${listing.title}</h4>

    <p>${listing.location}</p>

    <p>₹${listing.price}</p>
  </div>
  `
)
```

Popup Opens Automatically

```
JavaScriptconst popup = new mapboxgl.Popup({  
  offset: 25,  
})  
  
.setHTML(`<h4>${listing.title}</h4>`)  
  
.addTo(map);
```

Popup Styling

Inside `style.css`

```
CSS.mapboxgl-popup-content{  
  border-radius: 12px;  
  padding: 12px;  
  text-align:center;  
}
```

Full Flow

```
Coordinates  
↓  
Marker Created  
↓  
Popup Attached  
↓  
User Clicks Marker  
↓  
Popup Opens
```

Final Result

✅ Marker popup ✅ Dynamic listing data ✅ Professional UI ✅ Interactive map experience ✅ Airbnb-style location popup